

Skill 6

Task 1

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define MAX_COMMANDS 5
#define MAX_LENGTH 100
char history[MAX_COMMANDS][MAX_LENGTH];
int count = 0;
void storeCommand(char command[]) {
    if (count < MAX_COMMANDS) {
        strcpy(history[count], command);
        count++;
    } else {
        // Remove oldest command
        for (int i = 0; i < MAX_COMMANDS - 1; i++) {
            strcpy(history[i], history[i + 1]);
        }
        strcpy(history[MAX_COMMANDS - 1], command);
    }
}
void displayHistory() {
    if (count == 0) {
        printf("History is empty.\n");
        return;
    }
    printf("\nCommand History:\n");
    for (int i = 0; i < count; i++) {
        printf("%d %s\n", i + 1, history[i]);
    }
}
```

```

    }
}

void displayHistory() {
    if (count == 0) {
        printf("History is empty.\n");
        return;
    }
    printf("\nCommand History:\n");
    for (int i = 0; i < count; i++) {
        printf("%d %s\n", i + 1, history[i]);
    }
}

void retrieveCommand(int number) {
    if (number < 1 || number > count) {
        printf("Invalid history number.\n");
        return;
    }
    printf("Command: %s\n", history[number - 1]);
}

int main() {
    char command[MAX_LENGTH];
    int choice, number;
    while (1) {
        printf("\n--- COMMAND HISTORY ---\n");
        printf("1. Store Command\n");
        printf("2. Display History\n");
        printf("3. Retrieve Command\n");
        printf("4. Exit\n");
        printf("Enter choice: ");

```

```

        printf("4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        getchar();
        switch (choice) {
            case 1:
                printf("Enter command: ");
                fgets(command, MAX_LENGTH, stdin);
                command[strcspn(command, "\n")] = '\0';
                storeCommand(command);
                printf("Command stored.\n");
                break;
            case 2:
                displayHistory();
                break;
            case 3:
                printf("Enter history number: ");
                scanf("%d", &number);
                retrieveCommand(number);
                break;
            case 4:
                exit(0);
            default:
                printf("Invalid choice.\n");
        }
    }
    return 0;
}

```

OUTPUT:

```
root@balaji@BalajiPatil:~# nano Commands.c
root@balaji@BalajiPatil:~# gcc Commands.c -o Commands
root@balaji@BalajiPatil:~# ./Commands

--- COMMAND HISTORY ---
1. Store Command
2. Display History
3. Retrieve Command
4. Exit
Enter choice: 1
Enter command: Store Command
Command stored.

--- COMMAND HISTORY ---
1. Store Command
2. Display History
3. Retrieve Command
4. Exit
Enter choice: 3
Enter history number: 1
Command: Store Command

--- COMMAND HISTORY ---
1. Store Command
2. Display History
3. Retrieve Command
4. Exit
Enter choice: |
```

Task 2

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main() {
    int fd[2];
    pid_t pid;

    char message[] = "Hello from Parent Process";
    char buffer[100];

    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }

    pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid > 0) {
        // Parent process
        close(fd[0]);

```

```

        return 1;
    }

    if (pid > 0) {
        // Parent process
        close(fd[0]);

        write(fd[1], message, strlen(message) + 1);

        close(fd[1]);

        wait(NULL);

        printf("Parent: Data sent successfully.\n");
    }
    else {
        // Child process
        close(fd[1]);

        read(fd[0], buffer, sizeof(buffer));

        printf("Child received: %s\n", buffer);

        close(fd[0]);
    }

    return 0;
}

```

OUTPUT:

```

Enter pipeline commands separated by |:
Example: ls -l | grep txt | wc -l
Pipeline: ls -l | wc -l

Pipeline Structure:
1. ls -l
2. wc -l

Execution Order:
1 -> ls
2 -> wc

Executing pipeline...
111

Pipeline execution completed successfully.

```

Task 3

To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

```

GNU nano 7.2 command_pipe.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main() {
    int fd[2];
    pid_t p1, p2;
    if (pipe(fd) == -1) {
        perror("pipe");
        return 1;
    }
    p1 = fork();

    if (p1 < 0) {
        perror("fork");
        return 1;
    }
    if (p1 == 0) {
        // First command: ls
        close(fd[0]);
        dup2(fd[1], STDOUT_FILENO);
        close(fd[1]);
        execlp("ls", "ls", NULL);
        perror("execlp");
        exit(1);
    }
    p2 = fork();
    if (p2 < 0) {
        perror("fork");
    }
}

```

```

p2 = fork();
if (p2 < 0) {
    perror("fork");
    return 1;
}
if (p2 == 0) {
    // Second command: wc -l
    close(fd[1]);
    dup2(fd[0], STDIN_FILENO);
    close(fd[0]);
    execlp("wc", "wc", "-l", NULL);
    perror("execlp");
    exit(1);
}
// Paren
close(fd[0]);
close(fd[1]);
waitpid(p1, NULL, 0);
waitpid(p2, NULL, 0);
return 0;

```

Output:

```

root@balaji@BalajiPatil:~# ./new3
pipe created successfully.
11
data flow completed successfully.

```

TASK 4

To Support Multiple Pipes, Manage Descriptor Arrays, Launch Multiple Processes, Coordinate Execution, Cleanup Resources, Test Long Pipelines
--

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/wait.h>
int main() {
    pid_t pid;
    int fd;
    pid = fork();
    if (pid < 0) {
        perror("fork");
        return 1;
    }
    if (pid == 0) {
        // Open output file
        fd = open("output.txt",
                 O_WRONLY | O_CREAT | O_TRUNC,
                 0644);
        if (fd < 0) {
            perror("open");
            exit(1);
        }
        // Redirect stdout to file
        dup2(fd, STDOUT_FILENO);
        close(fd);
        // Execute command
        execlp("ls", "ls", NULL);
        perror("execlp");
        exit(1);
    }
}

```

Output:

```

root@balaji@BalajiPatil:~# nano redirection.c
root@balaji@BalajiPatil:~# gcc redirection.c -o redirection
root@balaji@BalajiPatil:~# ./redirection
Output redirected to output.txt
root@balaji@BalajiPatil:~# |

```